



Riesgo Sistémico en Ecosistemas de Software

Análisis del Grafo de Dependencias de PyPI mediante Cadenas de Markov y PageRank

Miramira Bellido, Rimsky Augusto

Machaca Muñiz, José Alejandro

Universidad Nacional de San Agustín

Escuela Profesional de Ciencias de la Computación

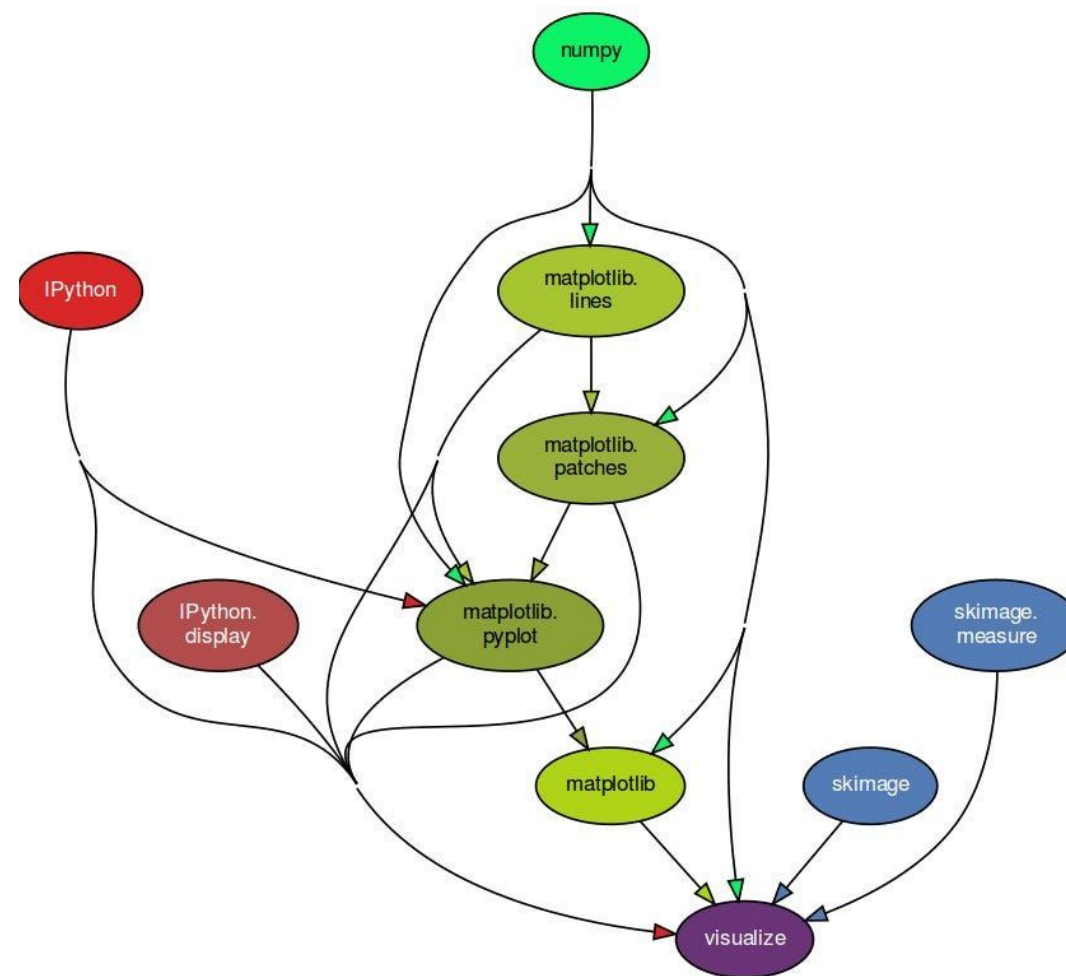
Arequipa, Perú – 2026

Capítulo I

Planteamiento del Problema

El Problema Estructural

- ⚠ **Supply Chain Risk:** El desarrollo moderno instala "árboles" opacos de dependencias de terceros.
- 🧩 **Propagación en Cascada:** Un fallo en un componente secundario (ej. *left-pad*, *log4j*) compromete a miles de proyectos que dependen de él.
- 🛡 **Deficiencia Reactiva:** Las herramientas actuales (Dependabot, Snyk) auditan vulnerabilidades conocidas (CVEs), pero ignoran la criticidad sistémica transitiva.
- ⚙ **Justificación:** Se requiere una métrica global que permita priorizar auditorías de seguridad *antes* de que ocurra el incidente.



? Problema General

¿En qué medida el PageRank calculado sobre el grafo de dependencias de PyPI, fundamentado en la distribución estacionaria de una Cadena de Markov, constituye un mejor predictor del impacto sistémico de un fallo que las métricas locales disponibles actualmente?

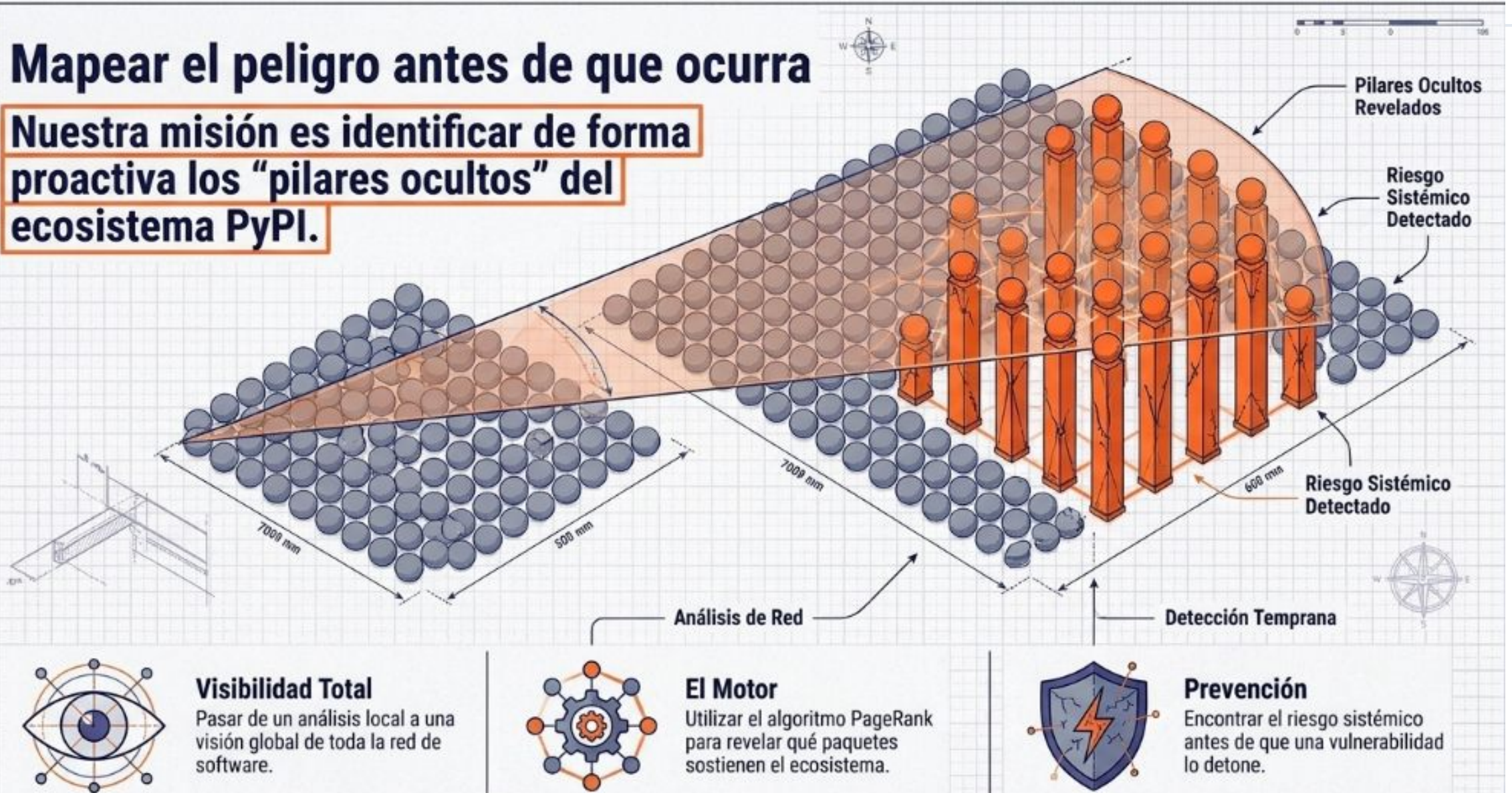
Problemas Específicos

- ▶ ¿Cuál es la estructura topológica del grafo y bajo qué condiciones garantiza ergodicidad?
- ▶ ¿Qué paquetes obtienen mayor PageRank y cómo difiere esto de métricas como el in-degree?
- ▶ ¿Cómo se distribuye este riesgo especialmente entre los clusters del ecosistema?
- ▶ ¿Cuántos proyectos dependen transitivamente del núcleo crítico?

Objetivos del Estudio

Mapear el peligro antes de que ocurra

Nuestra misión es identificar de forma proactiva los “pilares ocultos” del ecosistema PyPI.



Operacionalización de Variables

Tipo	Variable	Definición Operacional	Naturaleza
Dependiente	Vector PageRank (r^*)	Componente i -ésima del vector propio dominante de la Matriz de Google.	Cuantitativa Continua (Probabilidad)
Independiente	Factor Amortiguamiento (d)	Controla la mezcla entre la matriz A y la distribución uniforme E . Evaluado en $\{0.75, 0.85, 0.90, 0.95\}$.	Cuantitativa Continua
Independiente	In-degree (k^{in})	Número de paquetes que declaran al nodo como dependencia directa.	Cuantitativa Discreta
Independiente	Descargas Totales	Volumen mensual de descargas en PyPI (filtro ≥ 100).	Cuantitativa Discreta
Interviniente	Nodos Sumidero (Dangling)	Paquetes sin dependencias salientes ($outdeg = 0$). Corregidos en matriz A .	Categórica / Lógica

Capítulo II

Marco Teórico y Conceptual



Estructura Topológica

Kikas et al. (2017) analizaron npm y PyPI, encontrando redes libres de escala (cola pesada), pero limitándose a métricas puramente descriptivas y locales.



Propagación de Fallos

Decan et al. (2019) y Zimmermann (2019) midieron el impacto transitivo de vulnerabilidades en npm de forma **retrospectiva**, usando CVEs ya conocidos.



Brecha Identificada

Ningún estudio previo combinó Cadenas de Markov y PageRank para crear un índice estocástico, **proactivo** y estructural de riesgo sobre PyPI.

Matriz de Dependencias (H)

Dada la matriz de adyacencia L , definimos la matriz de transición sub-estocástica por columnas:

$$H_{ij} = \frac{L_{ij}}{\text{outdeg}(j)}$$

Genera columnas nulas para paquetes sumidero ($\text{outdeg}(j) = 0$).

Corrección de Sumideros (A)

Para restaurar la estocasticidad y evitar que la masa de probabilidad se pierda, aplicamos un vector indicador a :

$$A = H + \frac{1}{n} \mathbf{1} a^T$$

Garantiza que $\mathbf{1}^T A = \mathbf{1}^T$, induciendo teleportación uniforme al llegar a librerías base.

Matriz de Google $G(d)$

Introducimos el factor de amortiguamiento $d \in (0,1)$ para garantizar ergodicidad (Teorema de Perron–Frobenius):

$$G(d) = dA + \frac{1-d}{n}E$$

Método de la Potencia

Calculamos el vector estacionario r^* (PageRank) de forma iterativa, aprovechando la dispersión de H para ahorrar memoria RAM:

$$r^{(k+1)} = G(d) r^{(k)}$$



Dependencia Transitiva

Si A depende de B, y B de C, A depende transitivamente de C. Estas cadenas largas ocultan el riesgo real a las métricas locales.



Nodos Sumidero

Paquetes base sin dependencias salientes (ej. *numpy*). Representan barreras topológicas que requieren corrección matemática estocástica.



Centralidad Local vs Global

Local (*In-degree*) cuenta conexiones directas. Global (*PageRank*) pondera la importancia transitiva de quienes te conectan.

Capítulo III

Metodología de Investigación

Dataset: libraries.io

Extracción del snapshot masivo de dependencias open source (v1.6.0). Volumen original: >120 GB, 190 millones de aristas globales.

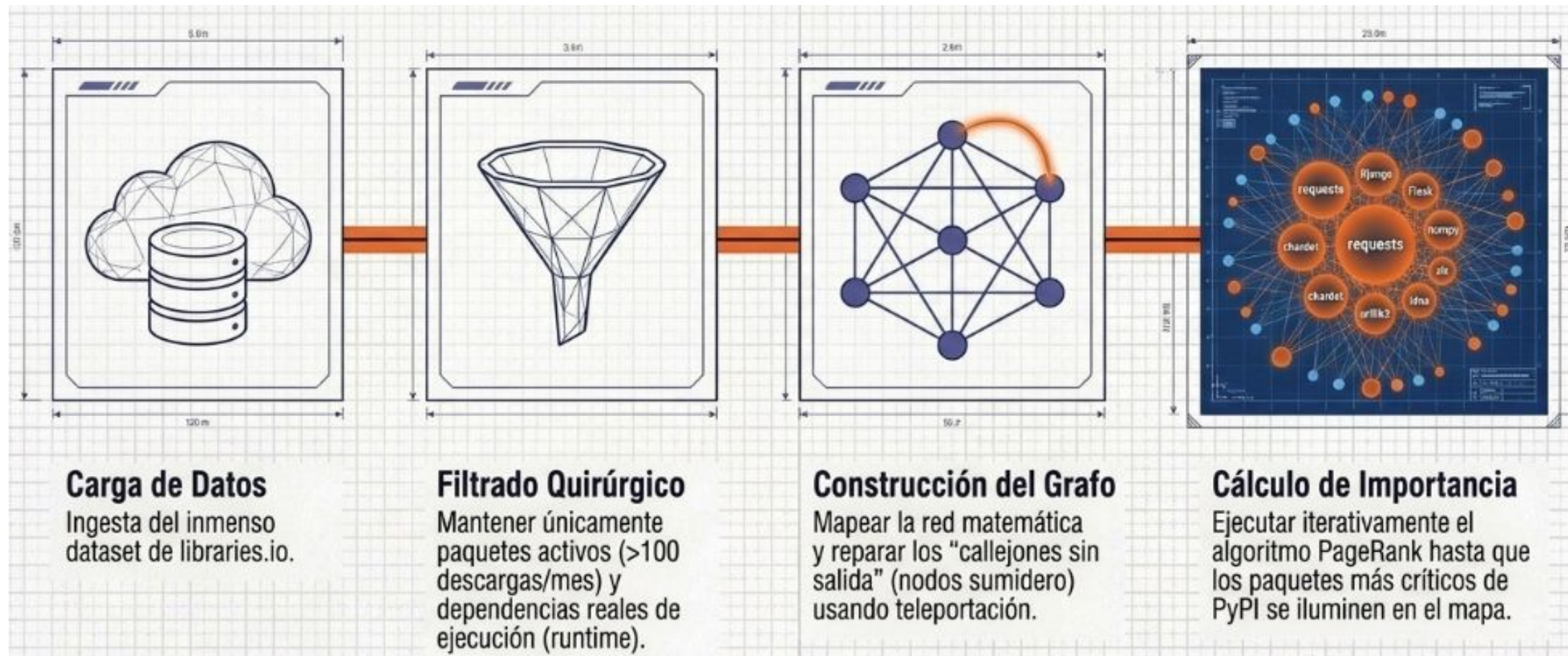
- ✓ **Filtro Ecosistema:** Exclusivamente PyPI.
- ✓ **Tipo de Arista:** Solo dependencias *runtime* (excluyendo development/optional).

Criterios de Inclusión

Para modelar con precisión entornos de producción corporativos, se depuró la red eliminando paquetes abandonados.

- 🔴 **Umbral Actividad:** Mínimo de 100 descargas mensuales históricas.
- 📊 **Muestra Final:** 849,606 paquetes activos que constituyen el grafo gigante analizado.

Pipeline de Análisis Computacional



Capítulo IV

Resultados y Análisis

4.1 Topología del Grafo de Dependencias

La distribución confirma una red libre de escala (scale-free). Destaca la proporción crítica de nodos sumidero (24.84%), requiriendo corrección estocástica obligatoria.

Métrica Topológica	Valor Obtenido (Ecosistema PyPI)
Número de nodos (paquetes activos)	849,606
Número de aristas (dependencias runtime)	6,976,793
Densidad $\delta = m/n^2$	9.66×10^{-6}
Exponente de ley de potencias (γ)	1.77
Paquetes sin dependencias salientes (Sumideros)	211,021 (24.84%)
Tamaño de la Componente Gigante (GSCC)	1 / 849,606 (0.00%)

4.2 Convergencia del Método de la Potencia

El algoritmo alcanzó la tolerancia de error ($L_1 < 10^{-8}$) exhibiendo un comportamiento de convergencia geométrica consistente con la teoría de Perron–Frobenius.

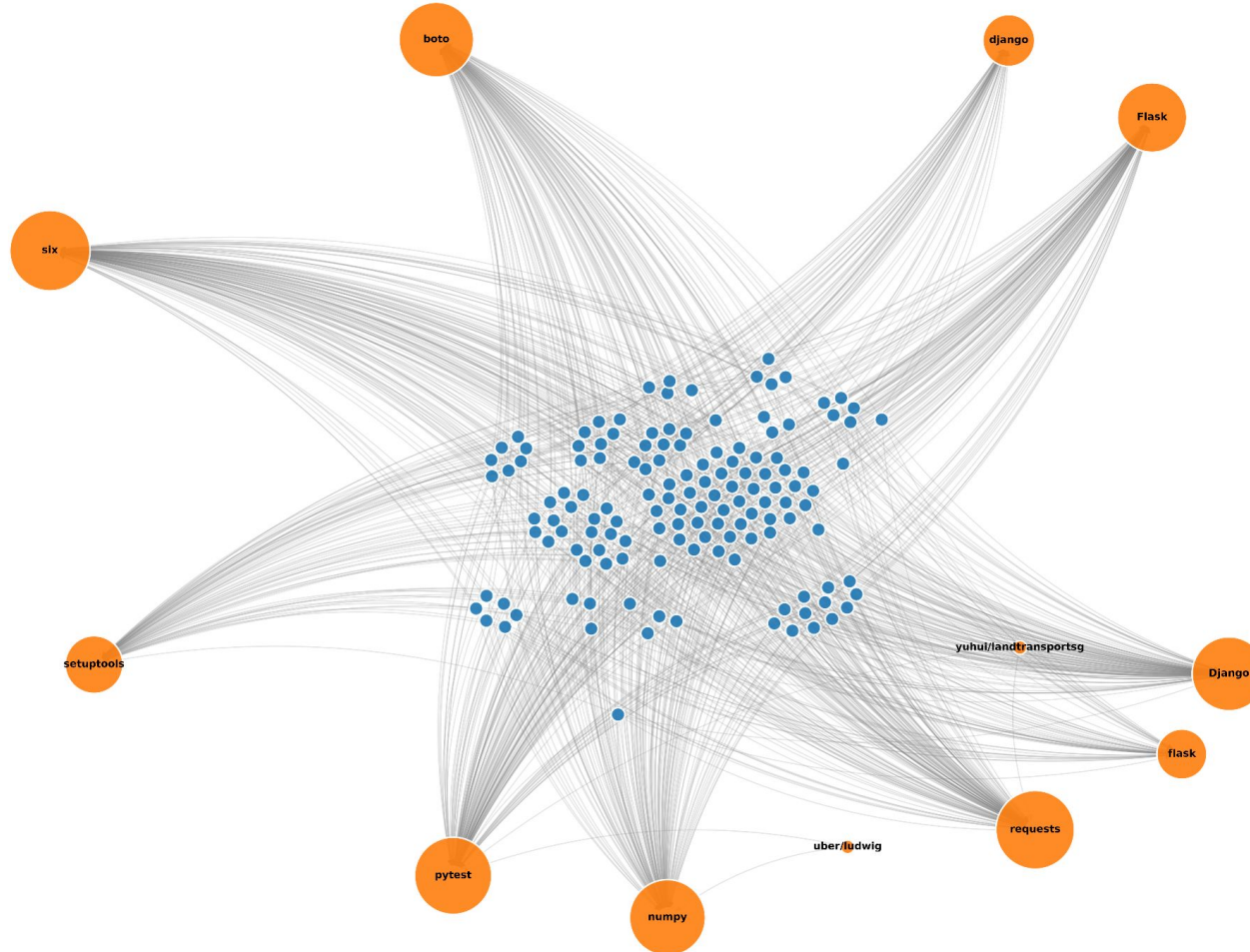
Factor Amortiguamiento (d)	Tasa Geométrica Teórica	Iteraciones Observadas
0.75	0.75	34
0.85 (Estándar de la industria)	0.85	43
0.90	0.90	49
0.95	0.95	57

4.3 El Núcleo Sistémico: Top 10 PageRank

Posición	Paquete Crítico	Cluster (Comunidad)	Categoría Principal
1	requests	2197	Librería HTTP / Red
2	Django	1	Framework Web
3	Flask	2	Framework Web (Micro)
4	numpy	3	Computación Científica
5	six	5	Compatibilidad Python 2/3
6	pytest	10	Framework de Testing
7	setuptools	6	Gestión de Paquetes
8	django (Alias)	1	Framework Web (Alias)
9	flask (Alias)	2197	Framework Web (Alias)
10	boto	1	SDK para AWS Cloud

4.3 El Núcleo Sistémico: Top 10 PageRank

Red de Impacto Local: Top Paquetes y Anomalías (Vecindario)



4.4 Divergencia Analítica: Riesgo Oculto

Detección de anomalías: Paquetes cuya criticidad sistémica es "invisible" para métricas locales como el in-degree, actuando como cuellos de botella transitivos críticos.

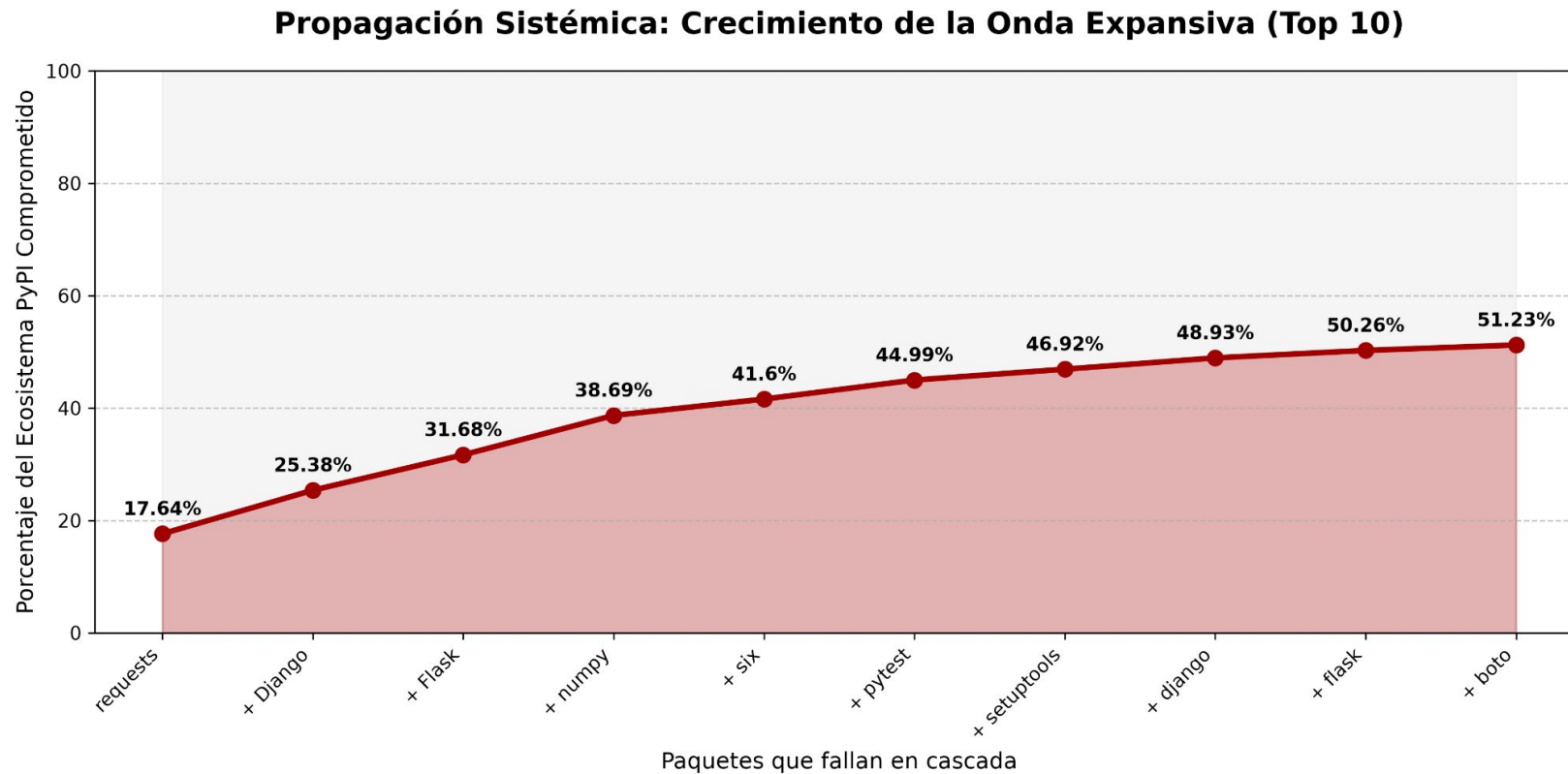
Paquete Anómalo	Rango PageRank	Rango In-Degree	Análisis de Centralidad Transitiva
larsjsol/shellpic	254,069	679,816	Nodo intermedio crítico en sub-cadenas largas.
uber/ludwig	373,810	799,557	Dependencia anidada en ecosistemas de IA robustos.
explosion/spacy-stanfordnlp	383,906	809,653	Componente clave en pipelines de NLP.

- **El Punto Ciego:** El In-degree (dependencias directas) ignora la red profunda.
- **El Hallazgo:** Paquetes "irrelevantes" saltan más de 400.000 posiciones al usar PageRank.
- **El Riesgo:** Actúan como puntos únicos de fallo invisibles para todo el ecosistema.

4.5 Análisis de Cobertura Transitiva (BFS Inverso)

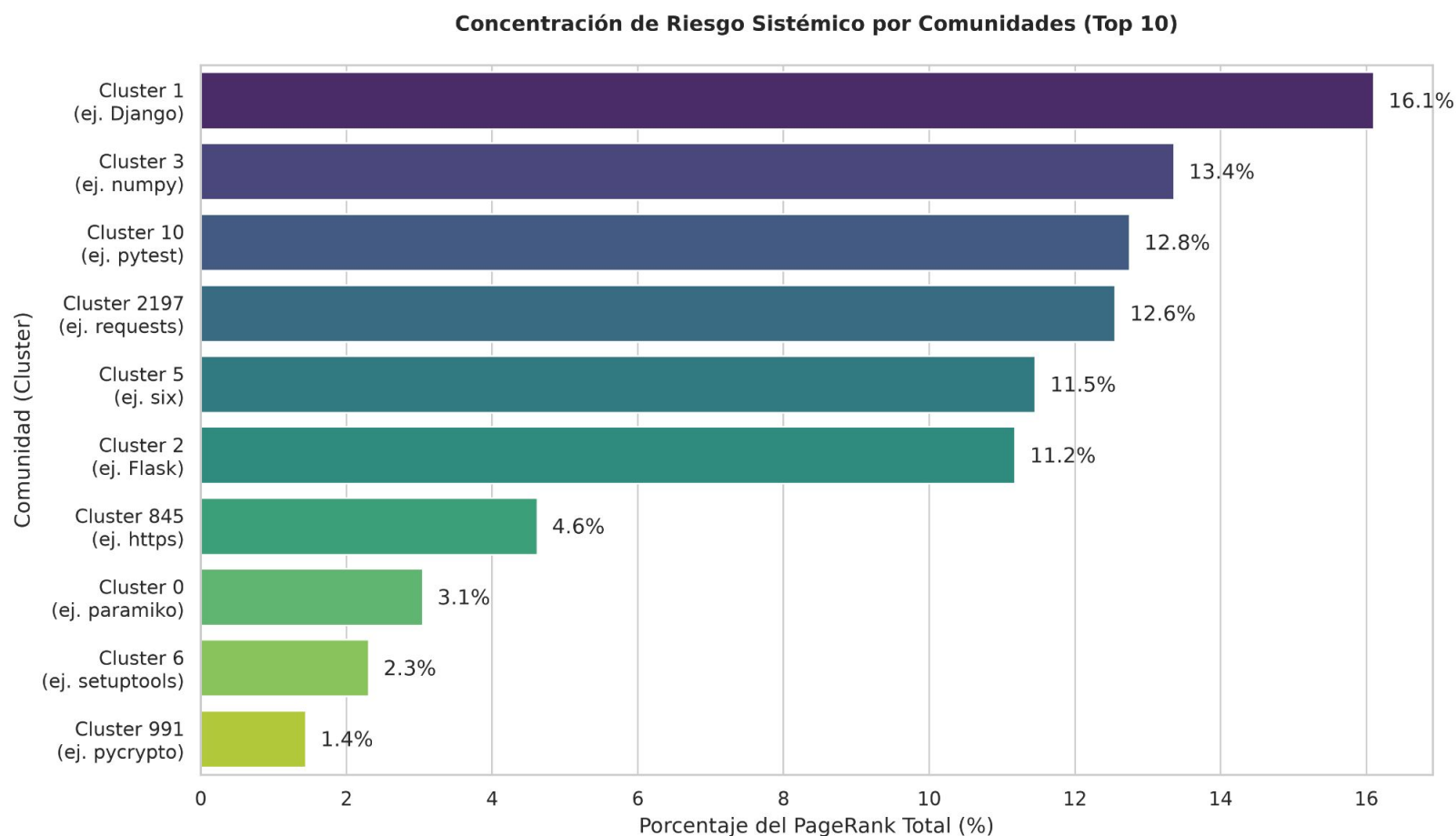
#	Paquete Base	Nodos (Individual)	% (Individual)	Nodos (Acumulada)	% (Acumulada)
1	requests	149,908	17.64%	149,908	17.64%
2	Django	89,587	10.54%	215,610	25.38%
3	Flask	83,653	9.85%	269,160	31.68%
4	numpy	85,718	10.09%	328,716	38.69%
5	six	119,929	14.12%	353,413	41.60%
6	pytest	60,596	7.13%	382,233	44.99%
7	setuptools	28,404	3.34%	398,654	46.92%
8	django	28,704	3.38%	415,701	48.93%
9	flask	31,439	3.70%	426,992	50.26%
10	boto	22,173	2.61%	435,219	51.23%

4.5 Análisis de Cobertura Transitiva (BFS Inverso)



4.6 Concentración de Riesgo por Comunidades

Algoritmo de Louvain: Los 5 principales clusters acaparan más del 66% del riesgo total. La red presenta una segregación semántica natural.



Capítulo V

Conclusiones y Trabajos Futuros

- ✓ **Validez del Modelo:** La formalización de PyPI como Cadena de Markov ergódica (Matriz G) generó un índice de riesgo sistémico superior y proactivo frente a herramientas reactivas.
- ✗ **Divergencia Estructural:** Aunque hay alta correlación ($\rho = 0.9896$) con el in-degree, se probaron casos de "riesgo oculto" en paquetes anidados (ej. *spacy-stanfordnlp*) invisibles para auditorías locales.
- ⚡ **Centralización Masiva:** La métrica de Cobertura Transitiva demostró que una falla en apenas 10 paquetes críticos comprometería directamente al **51.23%** de todo el ecosistema PyPI activo.
- 🗺️ **Segregación del Riesgo:** La red está sectorizada cualitativamente. Un ataque al Cluster 3 (IA) no derivaría necesariamente al Cluster 1 (Web), ofreciendo mapas tácticos para la ciberseguridad.
- 🌐 **Generalización:** El pipeline (Parquet, CSR, Louvain) es aplicable a npm, Maven o Cargo.

Análisis Dinámico y Temporal

- Evolucionar del modelo estático (snapshot) a un análisis temporal mes a mes.
- **Objetivo:** Identificar dependencias emergentes y predecir su consolidación como "cuellos de botella" antes de que se vuelvan sistémicas.

Ponderación de Riesgo en la Cadena de Markov

- Reemplazar la transición probabilística uniforme por aristas con pesos.
- **Objetivo:** Que el random walk asigne mayor probabilidad a rutas de software con mala salud de mantenimiento (bajo número de commits, pocos mantenedores, o bajo bus factor).

Integración Aplicada (Cybersecurity DevSecOps)

- Desarrollar un plugin para IDEs (VS Code) o herramientas CI/CD que utilice esta métrica para advertir a los desarrolladores en tiempo real sobre el riesgo sistémico de la librería que están a punto de instalar.

Referencias Bibliográficas

BIBLIOGRAFÍA

[1] M. Zimmermann, C.-A. Staicu, C. Tenny y M. Pradel, «Small World with High Risks: A Study of Security Threats in the npm Ecosystem,» en Proceedings of the 28th USENIX Security Symposium, USENIX Association, 2019, págs. 995–1010.

[2] A. Decan, T. Mens y P. Grosjean, «On the impact of security vulnerabilities in the npm package dependency network,» en Proceedings of the 16th International Conference on Mining Software Repositories (MSR), IEEE, 2019, págs. 181–191. doi: 10.1109/MSR.2019.00026

[3] National Institute of Standards and Technology (NIST), CVE-2021-44228 Detail, <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>, Log4Shell vulnerability, dic. de 2021.

[4] R. Kikas, G. Gousios, M. Dumas y D. Pfahl, «Structure and evolution of package dependency networks,» en Proceedings of the 14th International Conference on Mining Software Repositories (MSR), IEEE, 2017, págs. 102–112. doi: 10.1109/MSR.2017.55

[5] Tidelif, Libraries.io Open Source Repository and Dependency Metadata, Dataset de dependencias de paquetes PyPI, npm y otros ecosistemas, Zenodo, 2020. doi: 10.5281/zenodo.3626071
dirección: <https://doi.org/10.5281/zenodo.3626071>

[6] S. Brin y L. Page, «The anatomy of a large-scale hypertextual Web search engine,» en Proceedings of the 7th International Conference on World Wide Web, Elsevier, 1998, págs. 107–117. doi: 10.1016/S0169-7552(98)00110-X

[7] V. D. Blondel, J.-L. Guillaume, R. Lambiotte y E. Lefebvre, «Fast unfolding of communities in large networks,» Journal of Statistical Mechanics: Theory and Experiment, vol. 2008, n.o 10, P10008, 2008. doi: 10.1088/1742-5468/2008/10/P10008